

Aprendizado de Máquina B MLP

Lourenço R. G. Araújo - DEST
Pedro O.S. Vaz de Melo - DCC

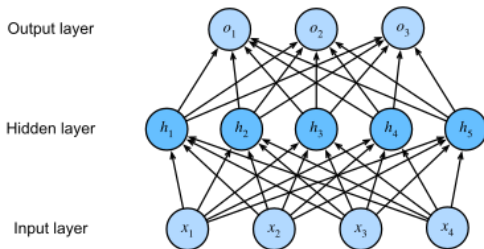
UFMG

Março de 2026

Multilayer Perceptron

- O que podemos fazer quando queremos separar padrões que não são linearmente separáveis?
- Exemplo de padrão não separável linearmente: XOR
- Combinando a saída de 2 perceptrons é possível separar o XOR - o algoritmo de treinamento do perceptron não funciona neste caso

Multilayer Perceptron



- Ideia básica: usamos neurônios para aprender, de forma não linear, representações latentes (ou projeções) a partir das nossas entradas, de forma que o problema fique linearmente separável
- Tanto os neurônios das camadas ocultas quanto da camada de saída são neurônios que combinam linearmente suas entradas e seus pesos e submetem a soma a uma função de ativação

Multilayer Perceptron

- A forma mais simples de combinar neurônios em camadas para aprender essas representações latentes consiste em construir camadas de neurônios totalmente conectadas.
- Para que a rede seja capaz de aprender representações não lineares, é necessário que as funções de ativação introduzam não linearidades.
- Para uma rede com uma camada oculta, dizemos que a rede possui 2 camadas.

Multilayer Perceptron

- O que acontece se usarmos funções de ativação lineares na camada oculta?
- Seja uma rede neural de 2 camadas
- Pensando em uma representação matricial, com entradas $\mathbf{X}_{n \times d}$, n neurônios na camada oculta com saída $\mathbf{H}_{n \times h}$ e saída $\mathbf{O}_{n \times k}$
- Temos um conjunto de pesos ligando as entradas à camada oculta $\mathbf{W}^{(1)}_{d \times h}$ e um viés $\mathbf{b}^{(1)}_{1 \times h}$ e um conjunto $\mathbf{W}^{(2)}_{h \times k}$ e um viés $\mathbf{b}^{(2)}_{1 \times k}$ que liga as saídas da camada oculta à camada de saída.

Multilayer Perceptron

- Se usarmos funções de ativação lineares:

$$\mathbf{H} = \mathbf{XW}^{(1)} + \mathbf{b}^{(1)}$$

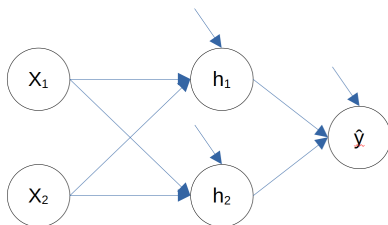
$$\mathbf{O} = \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}$$

- Ou seja:

$$\mathbf{O} = \mathbf{XW}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)} = \mathbf{XW} + \mathbf{b}$$

$$\text{se } \mathbf{W} = \mathbf{W}^{(1)}\mathbf{W}^{(2)} \text{ e } \mathbf{b} = \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

Multilayer Perceptron



$$h_1 = \varphi^{(1)}(w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + b_1^{(1)})$$

$$h_2 = \varphi^{(1)}(w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)})$$

$$\hat{y} = \varphi^{(2)}(w_{11}^{(2)}h_1 + w_{21}^{(2)}h_2 + b_1^{(2)})$$

Multilayer Perceptron

- Adotamos uma abordagem probabilística para as funções de ativação nas camadas de saída
- Para as camadas ocultas, é mais difícil ter uma resposta definitiva de qual função utilizar
- Abordagem ainda baseada em tentativa e erro, embora algumas funções sejam mais usadas que outras

Multilayer Perceptron

- A função ReLU (Rectified Linear Unit) é a recomendação padrão para redes neurais modernas

$$\varphi(x) = \max\{0, x\}$$

- Sempre que o neurônio está ativo, sua derivada é constante e igual a 1
- Função de ativação semelhante ao modelo linear, o que pode facilitar sua otimização
- Existem generalizações da função ReLU que tornam sua derivada diferente de zero quando $x < 0$: $\varphi(x) = \max\{\alpha, x\}$

Multilayer Perceptron

- Outras funções de ativação são usadas em camadas ocultas
- Antes das funções ReLU, a alternativa mais comum eram funções do tipo sigmoide: tangente hiperbólica ou sigmoide logística
- Um dos principais problemas envolvendo funções sigmoides é a saturação de $f(z)$ para valores de z muito positivos ou muito negativos
- Para usar funções do tipo sigmoide é necessário manter os valores de z pequenos

Multilayer Perceptron

- A inicialização dos pesos é, normalmente, feita com a utilização de heurísticas
- Valores iniciais excessivamente pequenos de pesos são problemáticos, assim como valores excessivamente grandes

Multilayer Perceptron

- Um dos principais atrativos teóricos de redes neurais do tipo MLP é a propriedade conhecida como Aproximação Universal
- Uma rede do tipo MLP, com pelo menos uma camada oculta e neurônios com função de ativação do tipo '*squashing*' é capaz de aproximar com acurácia arbitrária qualquer função mensurável de Borel de um espaço de dimensão finita pra outro (no nosso caso, consideramos uma função contínua em um subconjunto fechado e limitado de $\mathbb{R}^n$)

Multilayer Perceptron

- Redes MLP apresentam a propriedade de aproximação universal é extremamente poderoso, pois significa que, para qualquer função contínua que desejarmos aprender, uma rede neural será adequada, desde que suficientemente grande
- $y = f(x) + \epsilon$: somos capazes de aprender qualquer função $f(\cdot)$, independentemente de qualquer suposição prévia

Multilayer Perceptron

- O fato de existir uma rede neural capaz de aproximar tal função **não significa que**:
 - Somos capazes de encontrar tal função via otimização
 - Tal função é factível do ponto de vista prático (camada exponencialmente grande)
- Uma das vantagens de redes profundas é sua habilidade em representar certas famílias de funções de forma mais eficiente que redes rasas